머신러닝 기초 · 비지도학습

비지도학습
주성분 분석(PCA)

차원 축소 · 주성분 찾기 · 설명된 분산

v02 · 2026.10

목차

01 **차원 축소**
왜 줄이나 · 선택과 추출 · 쓰이는 곳

02 **PCA란**
분산이 가장 큰 방향 찾기

03 **주성분 찾기**
학생 6명 예시로 한 단계씩

04 **주성분의 설명력**
분산 비율 · 83.3% 계산

05 **실습과 정리**
스케일링 · 코드 · 개수 정하기 · t-SNE/UMAP

01 비지도학습 속 PCA

정답(레이블) 없이 데이터 구조를 찾는 세 갈래

1. **군집 (clustering)**
   비슷한 데이터끼리 묶는다 → 「비지도학습 기본」 덱

2. **변환 (transform)**
   데이터를 다루기 쉬운 새 표현으로 바꾼다. **차원 축소(PCA)**가 여기에 속한다 → 이 덱

3. **연관 (association)**
   함께 나타나는 항목의 규칙을 찾는다 → 「비지도학습 기본」 덱

01 차원 축소

무엇이고 왜 필요한가

차원 축소란

- 피처가 많은 데이터를 **더 적은 수의 피처**로 바꿔 새 데이터 세트를 만드는 것

차원의 저주

- 차원이 늘수록 데이터 포인트가 **공간에 희소하게(sparse) 흩어진다**
- 점 사이 거리가 비슷해져 거리 기반 방법이 약해진다

다중 공선성

- 피처가 많으면 서로 상관이 높은 피처가 생기기 쉽다
- 선형 회귀 같은 모델은 계수가 불안정해지고 예측 성능이 떨어질 수 있다

01 차원 축소 효과

해석 · 시각화

- 2~3차원으로 줄이면 그림으로 볼 수 있다

노이즈 감소

- 분산이 작은 방향을 버리면 잡음 일부가 줄어든다

메모리 · 속도

- 피처 수가 줄어 계산이 가벼워진다

성능

- 과적합이 줄어 성능이 좋아지기도 한다 (**항상은 아님**)

01 차원 축소 방법

피처 선택 vs 피처 추출

피처 선택 (feature selection)

- 기존 피처 중 **일부만 고른다**
- 불필요하거나 겹치는 피처를 뺀다

피처 추출 (feature extraction)

- 기존 피처를 섞어 **새 피처를 만든다**
- 단순 압축이 아니라 특징을 더 잘 설명하는 다른 공간으로 옮긴다

대표 알고리즘 (추출)

- PCA · TruncatedSVD · NMF (scikit-learn decomposition 모듈)

01 차원 축소가 쓰이는 곳

1. **이미지**
   픽셀이 매우 많은 이미지에서 잠재된 특징을 뽑는다

2. **텍스트**
   문서-단어 행렬에서 숨은 의미(주제)를 뽑는다 (LSA: TruncatedSVD)

3. **추천 시스템**
   사용자-아이템 행렬을 작은 잠재 요인으로 나타낸다

02 PCA (주성분 분석)

Principal Component Analysis

정의

- 여러 변수 사이의 상관관계를 이용해, **분산이 가장 큰 방향**부터 차례로 서로 **직교하는** 새 축(주성분)을 찾는 대표적인 차원 축소 기법

어떻게 줄이나

- 앞쪽 주성분 몇 개만 남기고 나머지 축을 버린다
- 남긴 축이 원래 데이터의 분산(정보)을 얼마나 담는지로 개수를 정한다

참고: scikit-learn User Guide 2.5.1 (확인일 2026-10-13)

03 주성분 찾기: 전체 흐름

다음 장부터 학생 6명 예시로 한 단계씩

STEP 1 중심 이동

- 데이터 중심(평균)을 원점(0,0)으로 옮긴다

STEP 2 PC1 찾기

- 원점을 지나는 직선 중, 사상한 점들의 거리 제곱합(SS) = 분산이 **최대**인 직선

STEP 3 PC2 찾기

- PC1에 **수직**인 방향

STEP 4 새 좌표로 표현

- 각 점을 PC1 · PC2에 사상하고 PC1이 가로축이 되도록 회전 → 각 점을 **(PC1, PC2) 새 좌표**로 나타낸다

03 예시 데이터: 학생 6명의 국어 · 영어 점수

2개 피처 = 2차원 데이터에서 주성분 찾기

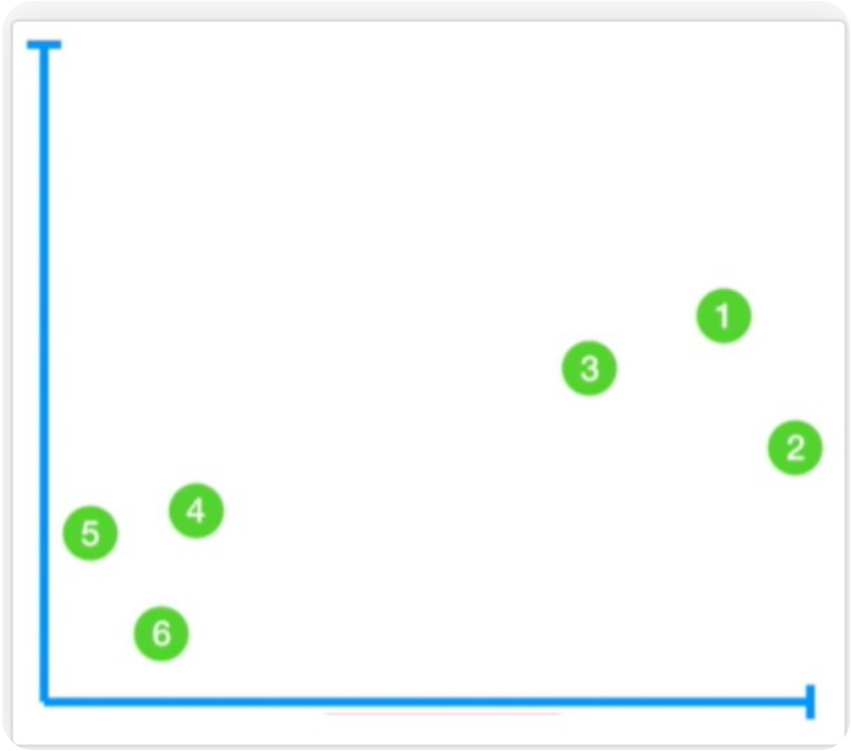
학생	국어	영어
학생1	10	6
학생2	11	4
학생3	8	5
학생4	3	3
학생5	2	2.8
학생6	1	1

샘플
학생 한 명 = 한 줄 (6개)

피처
국어 · 영어 점수 2개 = 2차원

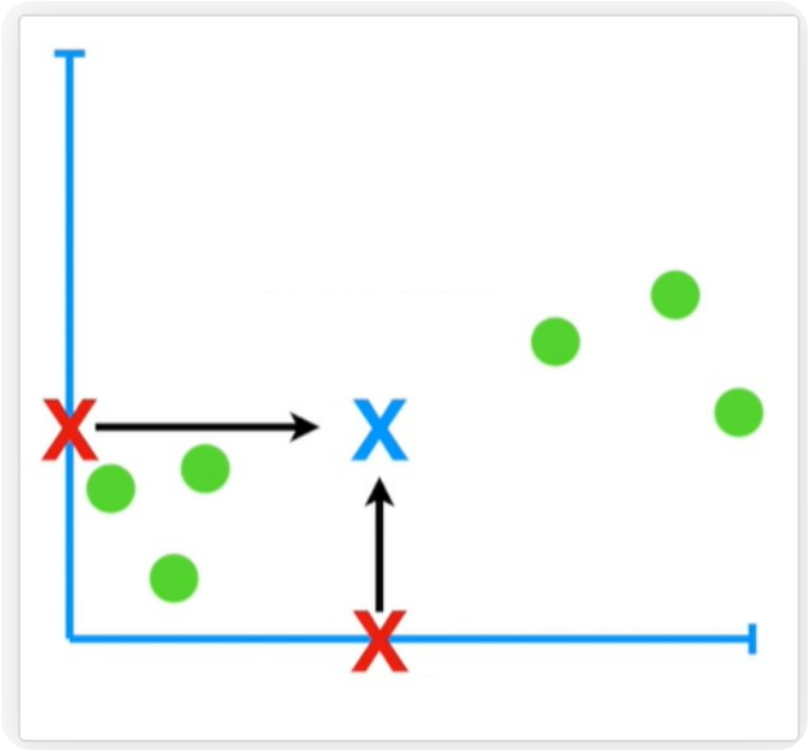
예시 데이터 (v01 덱의 값)

03 주성분 찾기 ① 데이터를 점으로



(A) 6명을 점으로 찍기

가로 국어 · 세로 영어

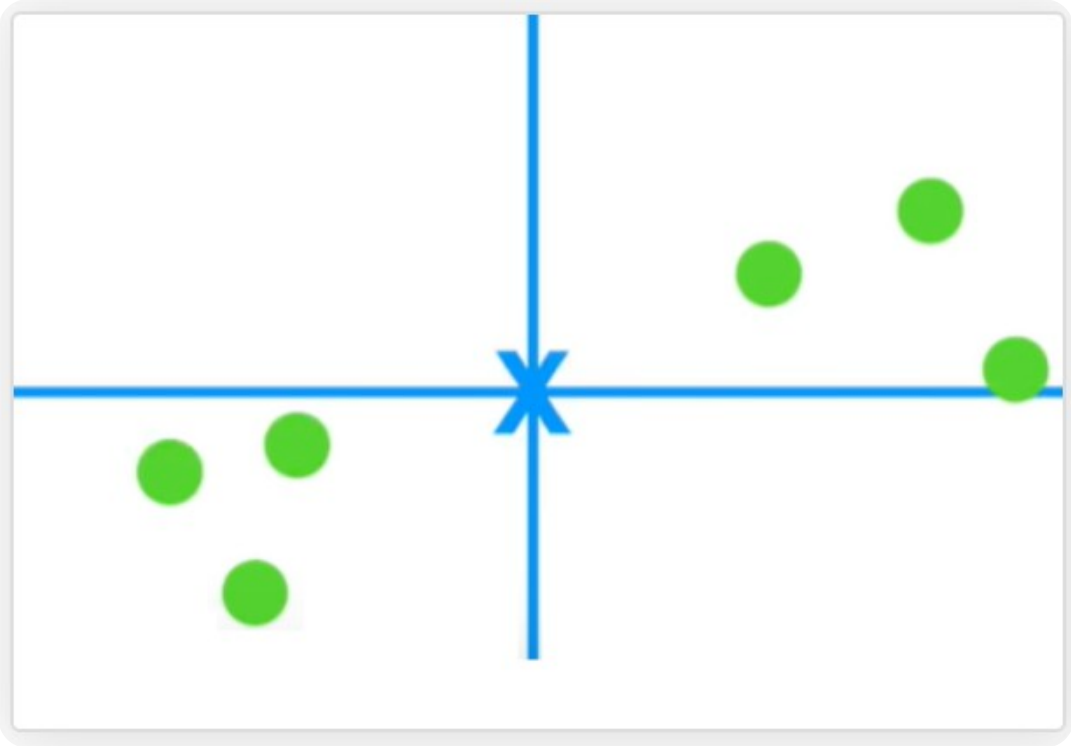


(B) 데이터 중심 정하기

국어 평균 · 영어 평균의 위치

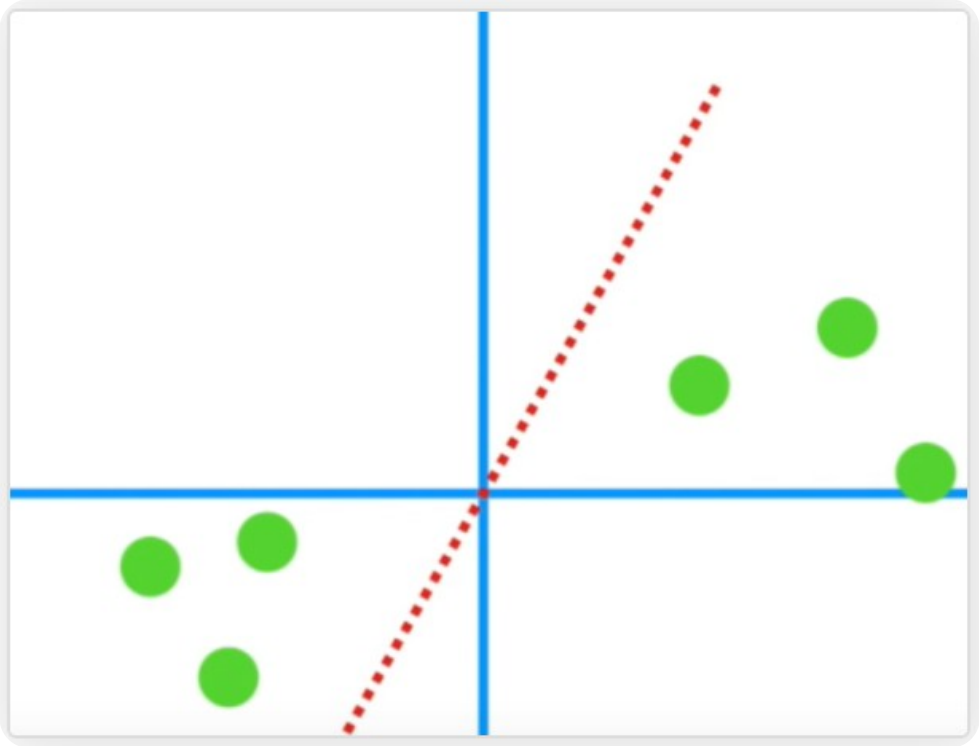
참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: angeloyeo.github.io)

03 주성분 찾기 ② 중심 이동



(C) 중심을 원점(0,0)으로

모든 점에서 평균을 뺀다

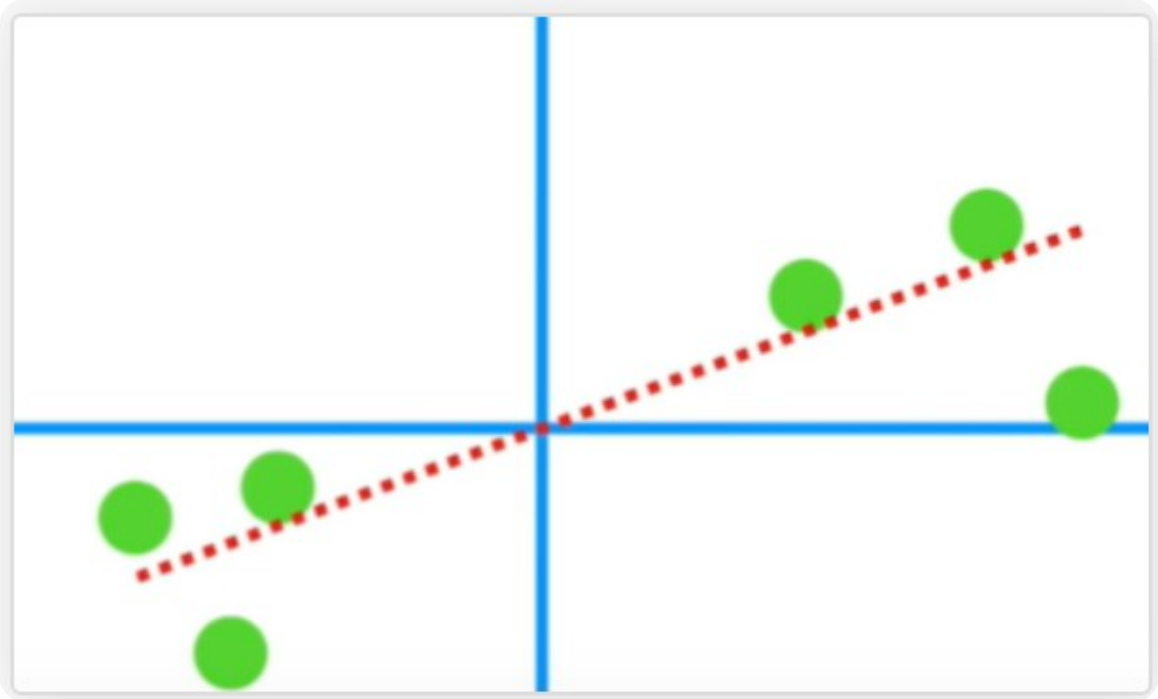


(D) 원점을 지나는 직선 하나

아무 방향에서 시작해 돌려 본다

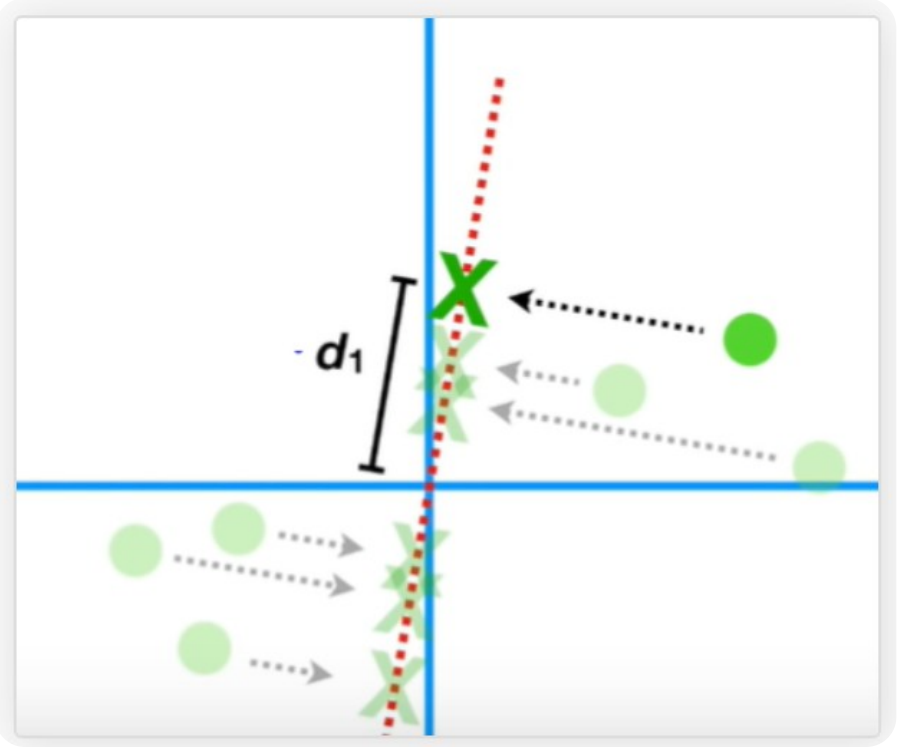
참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: angeloyeo.github.io)

03 주성분 찾기 ③ 직선에 사상하기



(E) 데이터에 가장 잘 맞는 직선

직선을 돌려 가며 찾는다



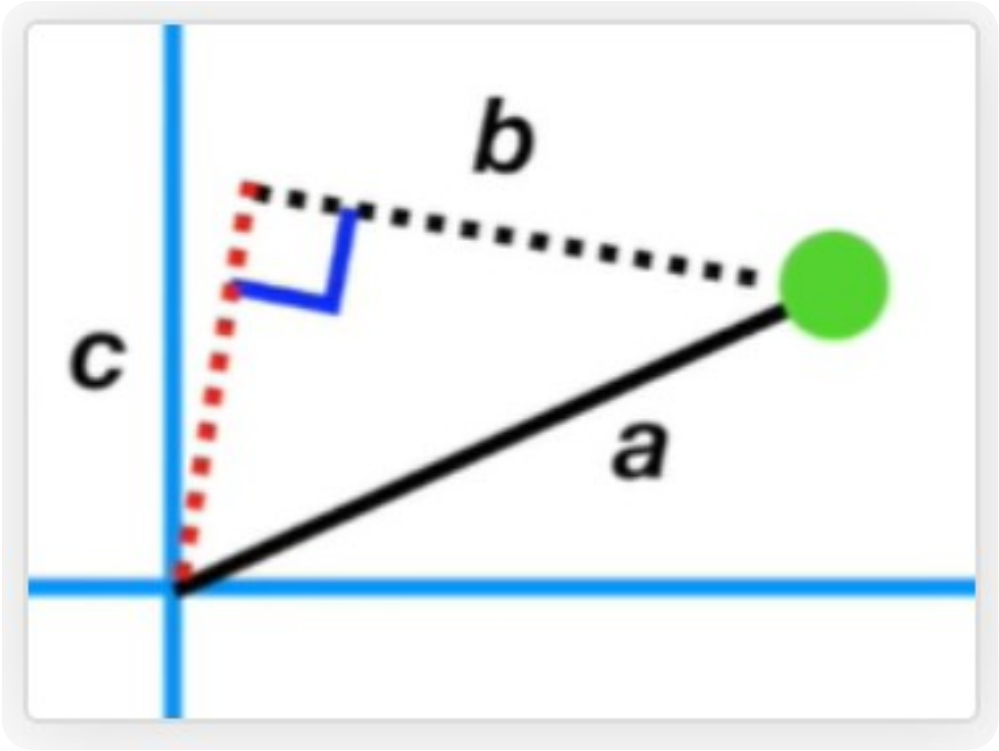
(F) 각 점을 직선에 수직으로 사상

원점~사상점 거리(d)의 **제곱합이 최대**인 직선

참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: angeloyeo.github.io)

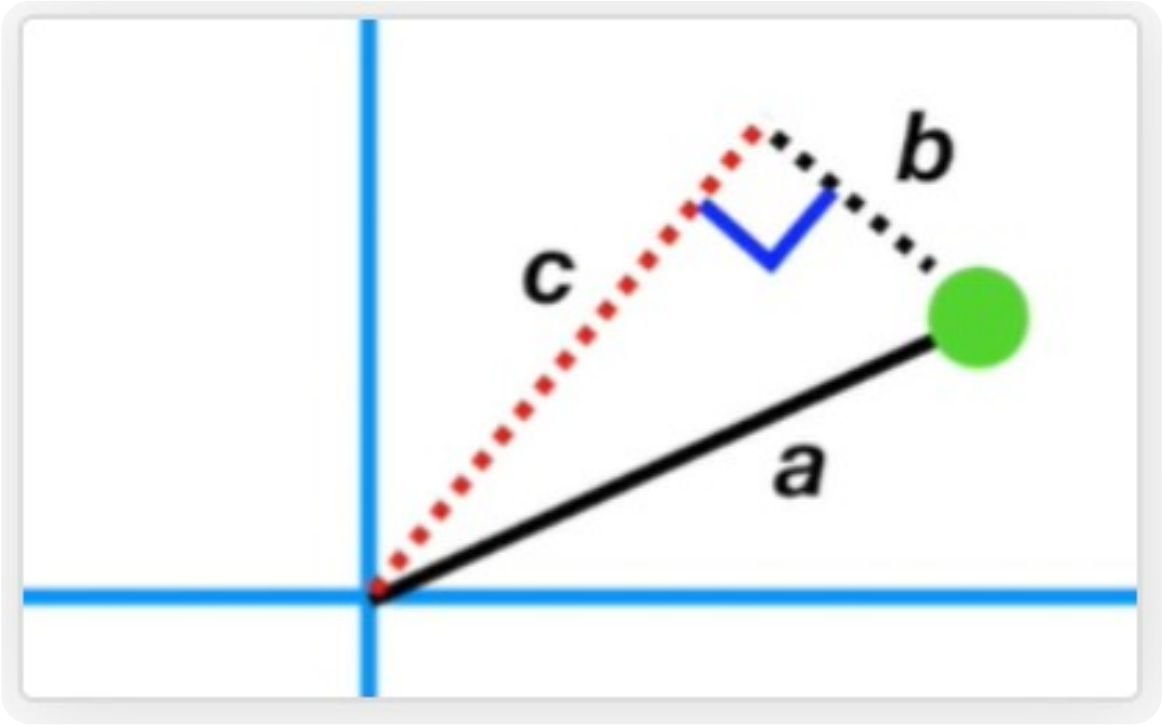
03 주성분 찾기 ④ 원리: 피타고라스 정리

$a^2 = b^2 + c^2$, a는 고정



(A) $a^2 = b^2 + c^2$

a: 원점~점, b: 점~직선, c: 원점~사상점

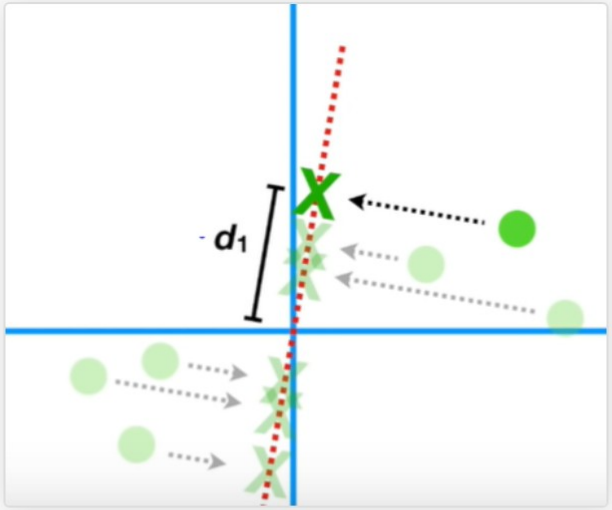


(B) c가 커지면 b는 작아진다

c^2 합 최대 = 직선까지 거리 b^2 합 최소

참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: angeloyeo.github.io)

03 주성분 찾기 ⑤ SS가 최대인 직선



$d_1^2 + d_2^2 + d_3^2 + d_4^2 + d_5^2 + d_6^2$ = sum of squared distances = SS(distances)

SS (sum of squared distances)

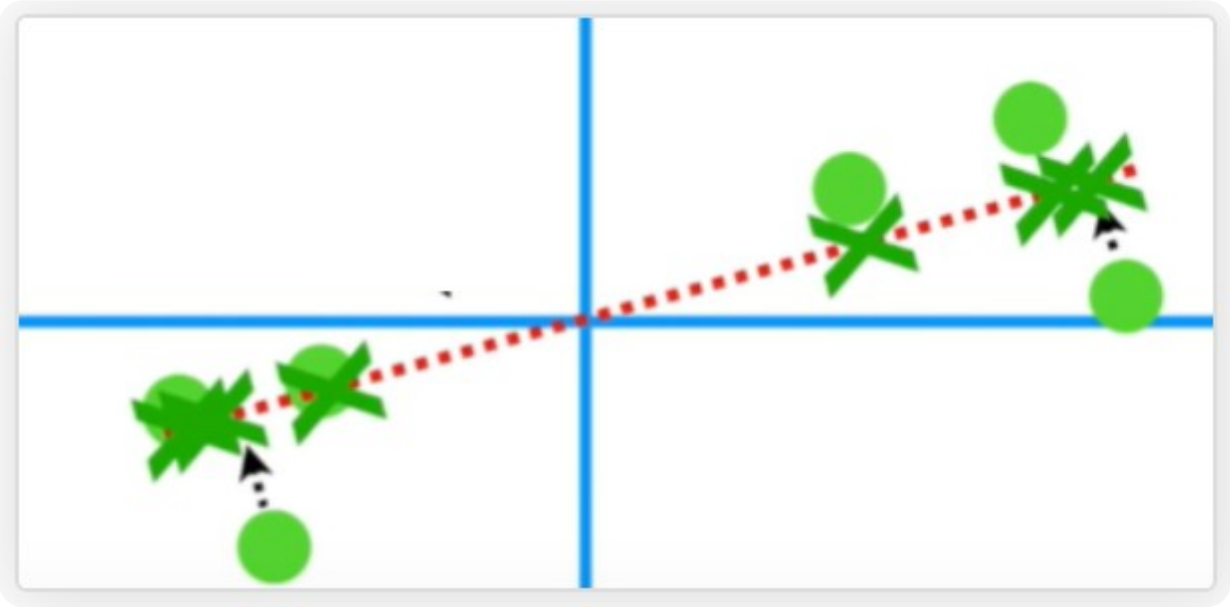
- 원점에서 6개 사상점까지 거리의 **제곱합**

SS 최대 = 분산 최대

- SS가 가장 큰 직선이 데이터에 가장 잘 맞는다
- 분산이 최대이면 정보 손실이 최소

참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: angeloyeo.github.io)

03 주성분 찾기 ⑥ 첫 번째 주성분 PC1

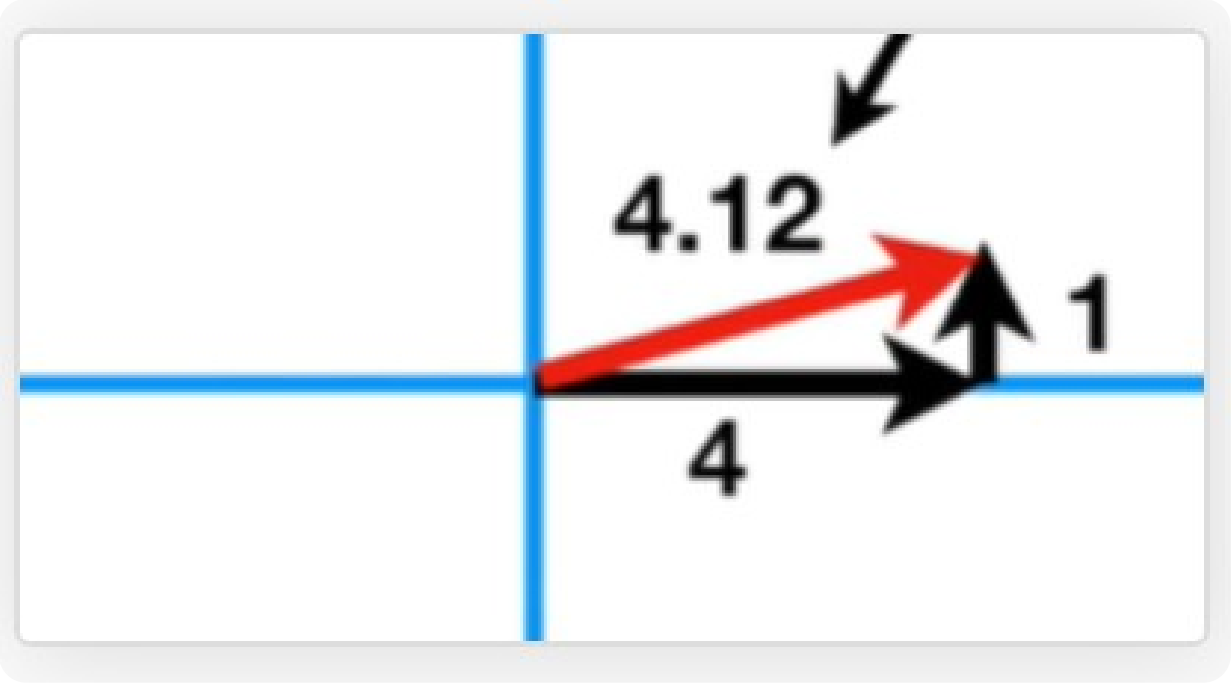


PC1

- SS가 최대인 이 직선이 **PC1(주성분 1)**
- 6개 데이터를 가장 잘 설명하는 방향

참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: bkshin.tistory.com)

03 주성분 찾기 ⑦ PC1의 방향



예시 그림의 기울기 0.25

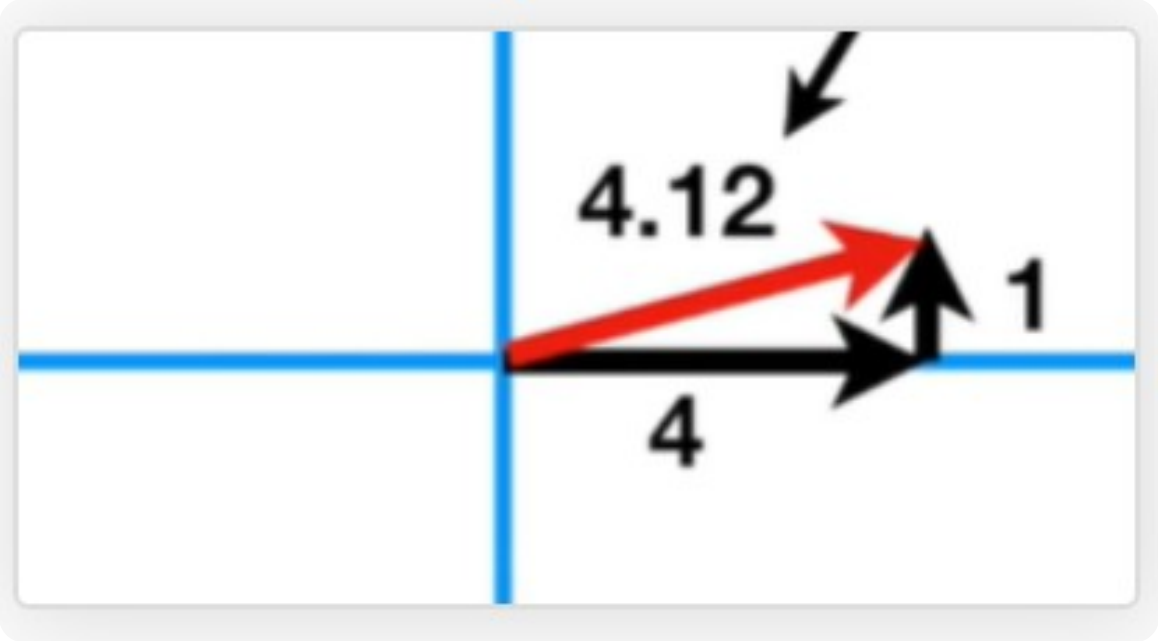
- 가로 4, 세로 1 → 기울기 1/4
- PC1 = **국어 4 : 영어 1**로 섞은 방향

주의

- 그림 값은 설명용 예시다. 앞 장 표 값으로 직접 계산한 기울기와는 다를 수 있다

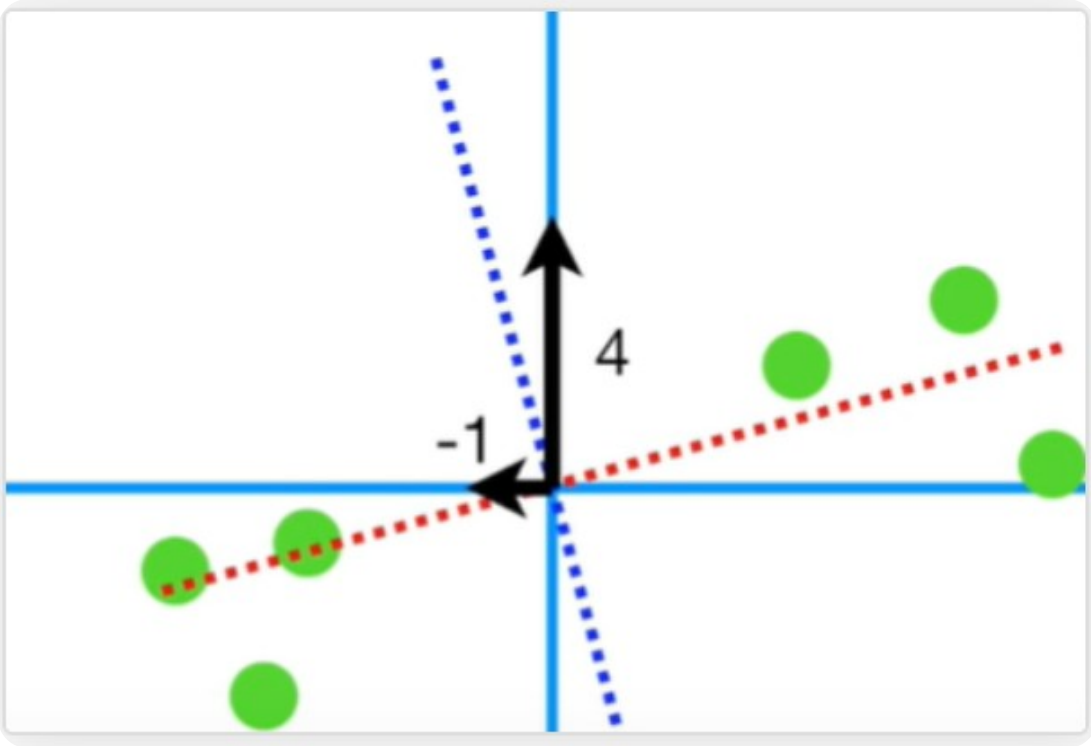
참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: bkshin.tistory.com)

03 주성분 찾기 ⑧ 두 번째 주성분 PC2



PC1 기울기 0.25 (예시 그림)

가로 4, 세로 1

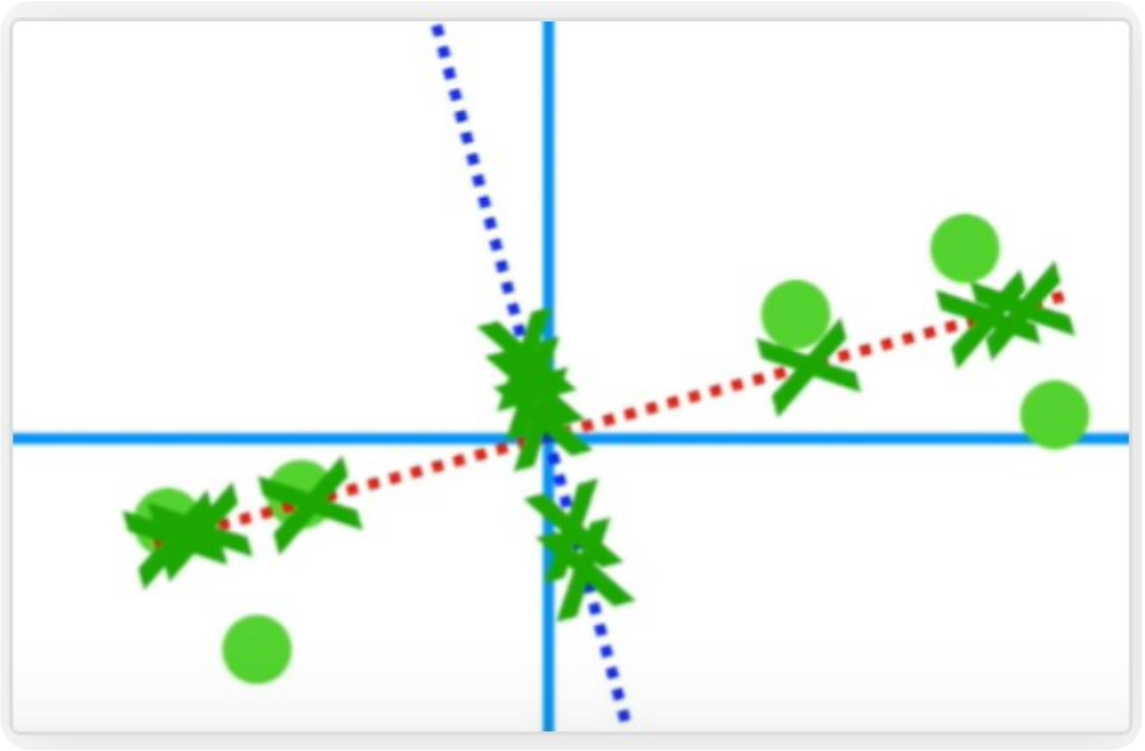


PC2 = PC1에 수직

기울기 -4 (0.25 × -4 = -1)

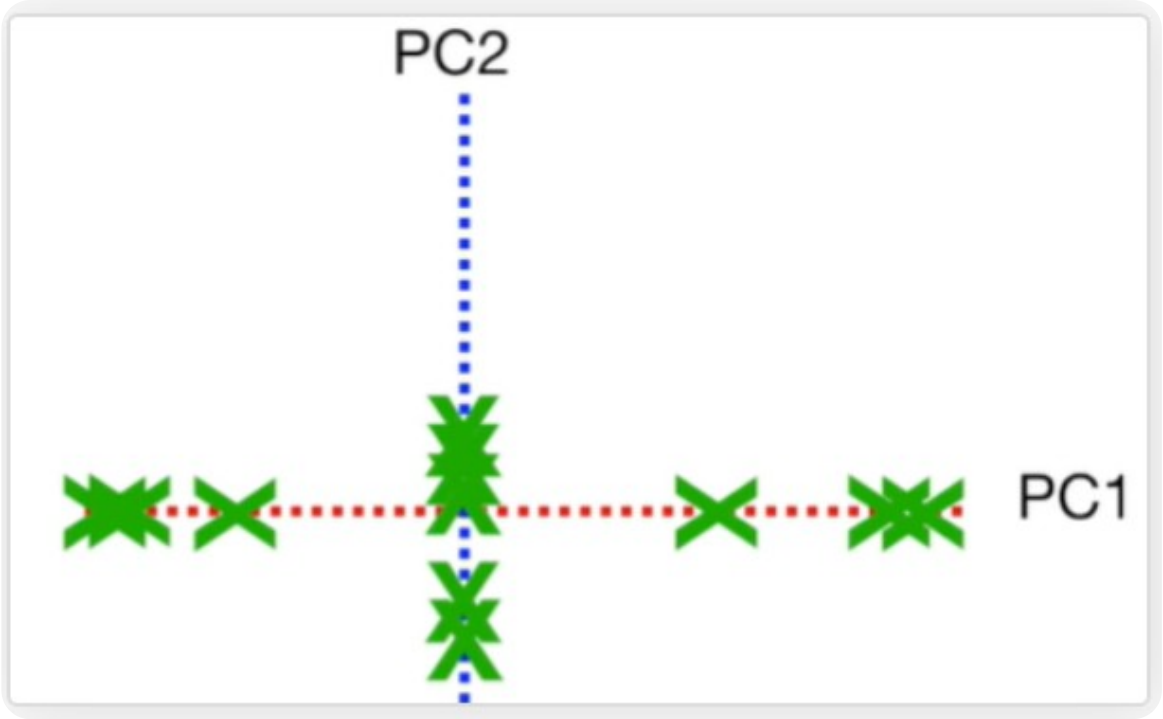
참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: bkshin.tistory.com)

03 주성분 찾기 ⑨ 사상과 회전



(A) PC1 · PC2에 사상

각 점을 두 주성분 위로 내린다

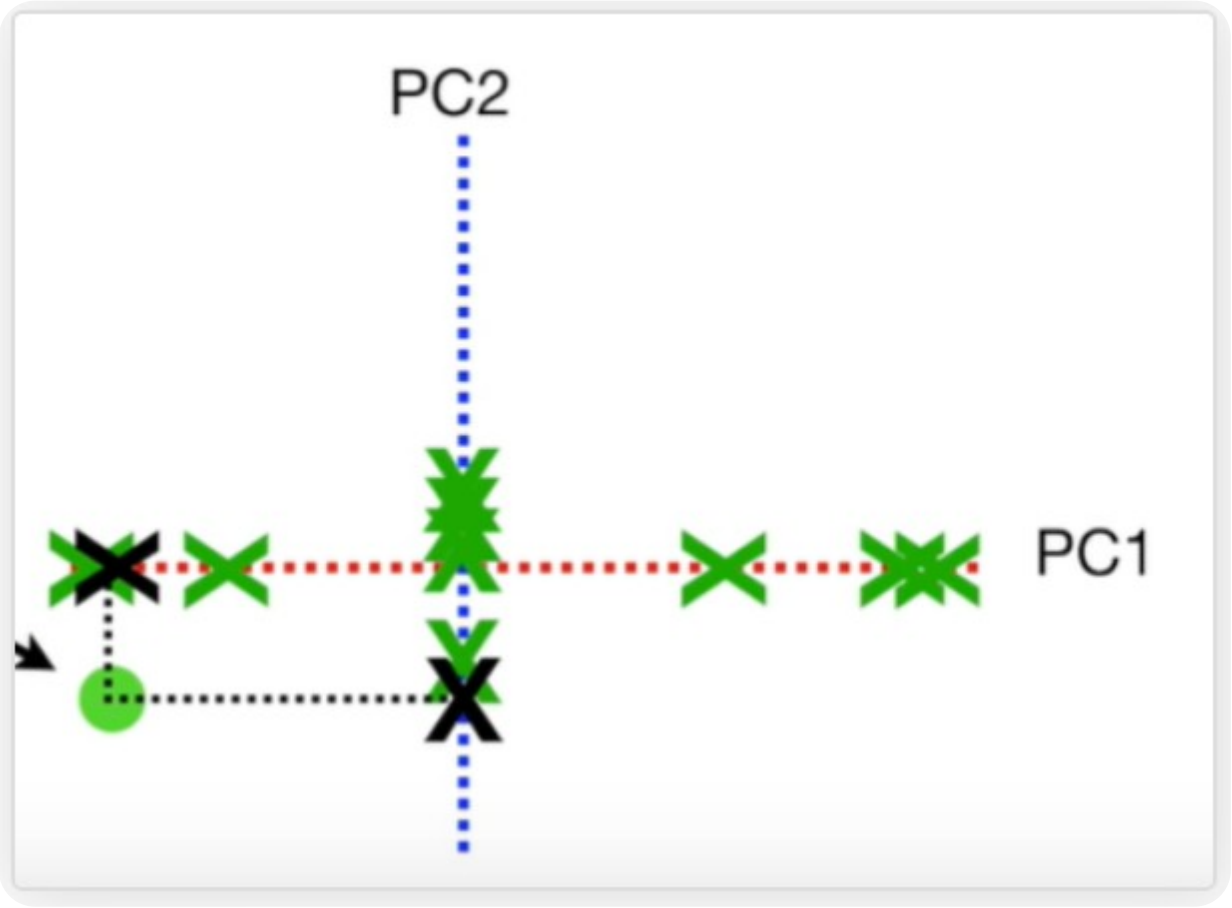


(B) PC1을 가로축으로 회전

PC1이 x축과 수평이 되도록

참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: bkshin.tistory.com)

03 주성분 찾기 ⑩ 새 좌표로 나타내기



참고: 원 출처 미확인 그림 (v01 노트의 참고 사이트: bkshin.tistory.com)

새 좌표

- 각 점을 **(PC1, PC2) 새 좌표**로 나타낸다
- PC1 위 사상값 = 가로 좌표, PC2 위 사상값 = 세로 좌표

모든 점에 반복

- 6개 점 모두 같은 방법으로 → PCA 변환 완료
- PC2를 버리면 1차원으로 축소

04 주성분의 설명력

▶ 분산 = SS / (n−1)

(A) SS를 n-1로 나누어 variation으로 구할 수 있다.

(B) PC1, PC2의 분산

$$\frac{SS(distances\ for\ PC1)}{n-1} = \text{Variation for PC1}$$

$$\frac{SS(distances\ for\ PC2)}{n-1} = \text{Variation for PC2}$$

04 주성분의 설명력

▶ 주성분 2개일 때 설명 비율

$$\frac{SS(distances\ for\ PC1)}{n-1} = \text{Variation for PC1}$$

$$\frac{SS(distances\ for\ PC2)}{n-1} = \text{Variation for PC2}$$

(A) PC1의 분산이 15, PC2의 분산이 3일 때, PC1은 $\frac{15}{18} = 0.83 = 83\%$

(B) PC1의 분산이 15, PC2의 분산이 3일 때, PC2은 $\frac{3}{18} = 0.17 = 17\%$

04 주성분의 설명력

▶ 주성분 4개일 때 설명 비율

(A) PC1~PC4 분산이 15, 5, 3, 1일 때, PC1은 $\frac{15}{24} = 0.625 = 62.5\%$

(B) PC1~PC4 분산이 15, 5, 3, 1일 때, PC2은 $\frac{5}{24} = 0.21 = 21\%$

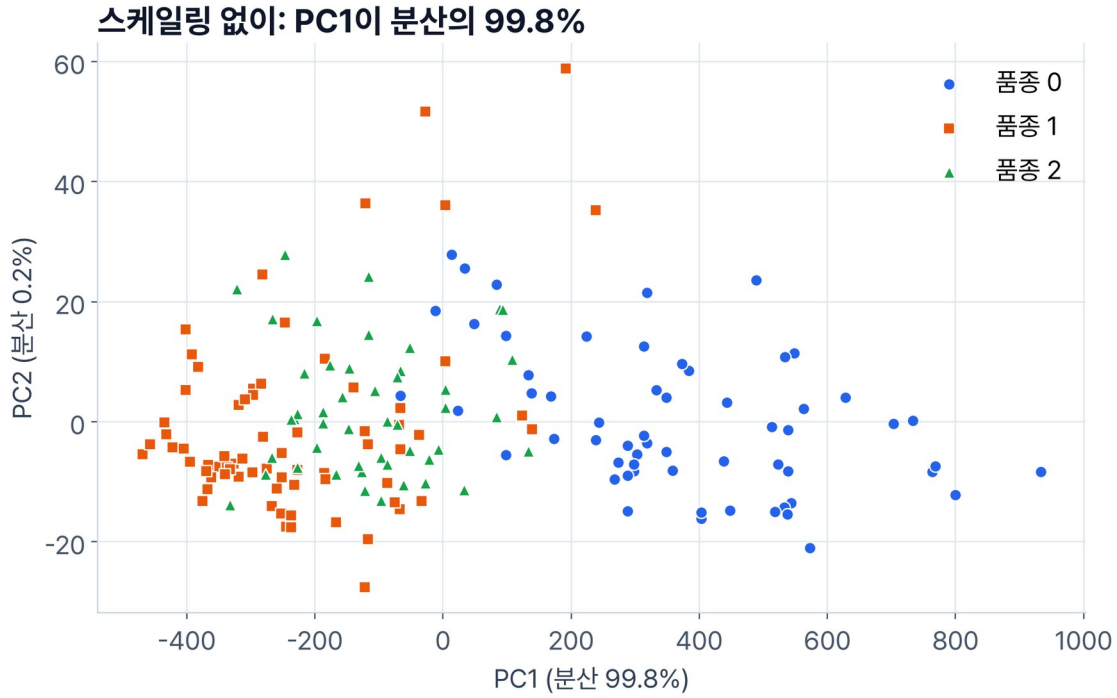
(C) PC1~PC4 분산이 15, 5, 3, 1일 때, PC3은 $\frac{3}{24} = 0.125 = 12.5\%$

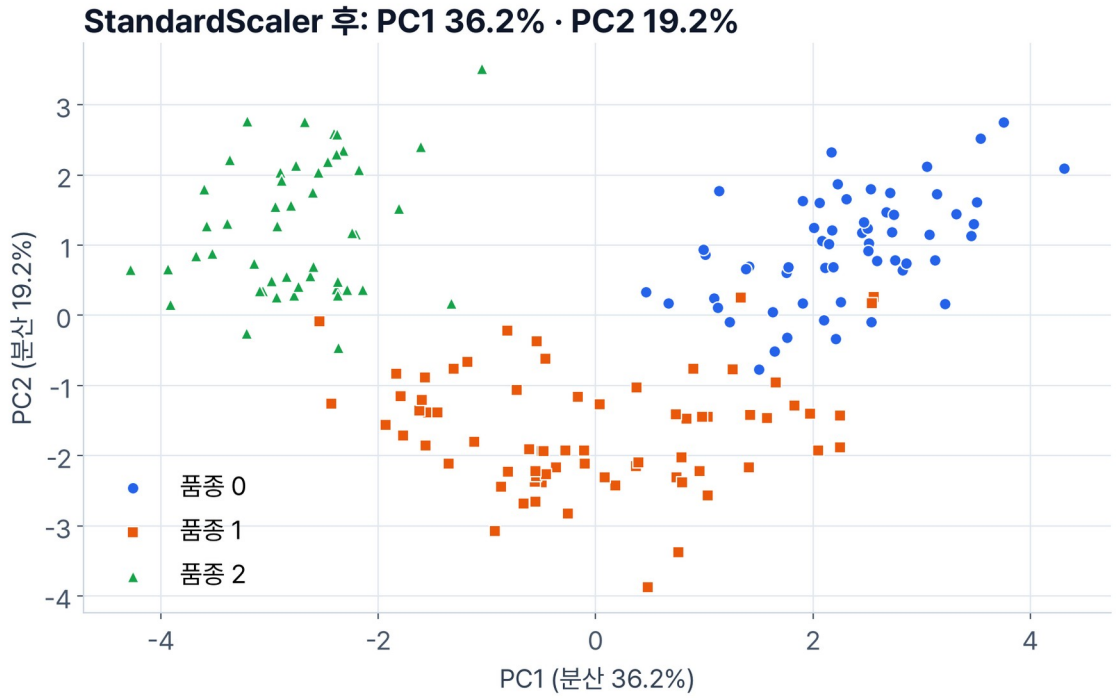
(D) PC1~PC4 분산이 15, 5, 3, 1일 때, PC4은 $\frac{1}{24} = 0.04 = 4\%$

PC1과 PC2 두 주성분으로 전체 분산의 83.3%(= 20/24)를 설명할 수 있다.

05 스케일링이 필요한 이유

단위가 큰 피처 하나가 주성분을 독차지한다





스케일링 없이: PC1 99.8%

값이 큰 proline 한 피처가 PC1 방향을 거의 혼자 정한다

StandardScaler 후: 36.2% · 19.2% · 11.1%

여러 피처가 고르게 반영되고 품종이 더 잘 갈린다

데이터: scikit-learn load_wine (178개 × 13피처), 품종별 색 · 모양, scikit-learn 1.9.1 실측 2026-10-13

05 scikit-learn으로 PCA

스케일링 → PCA → 설명 비율 확인

코드

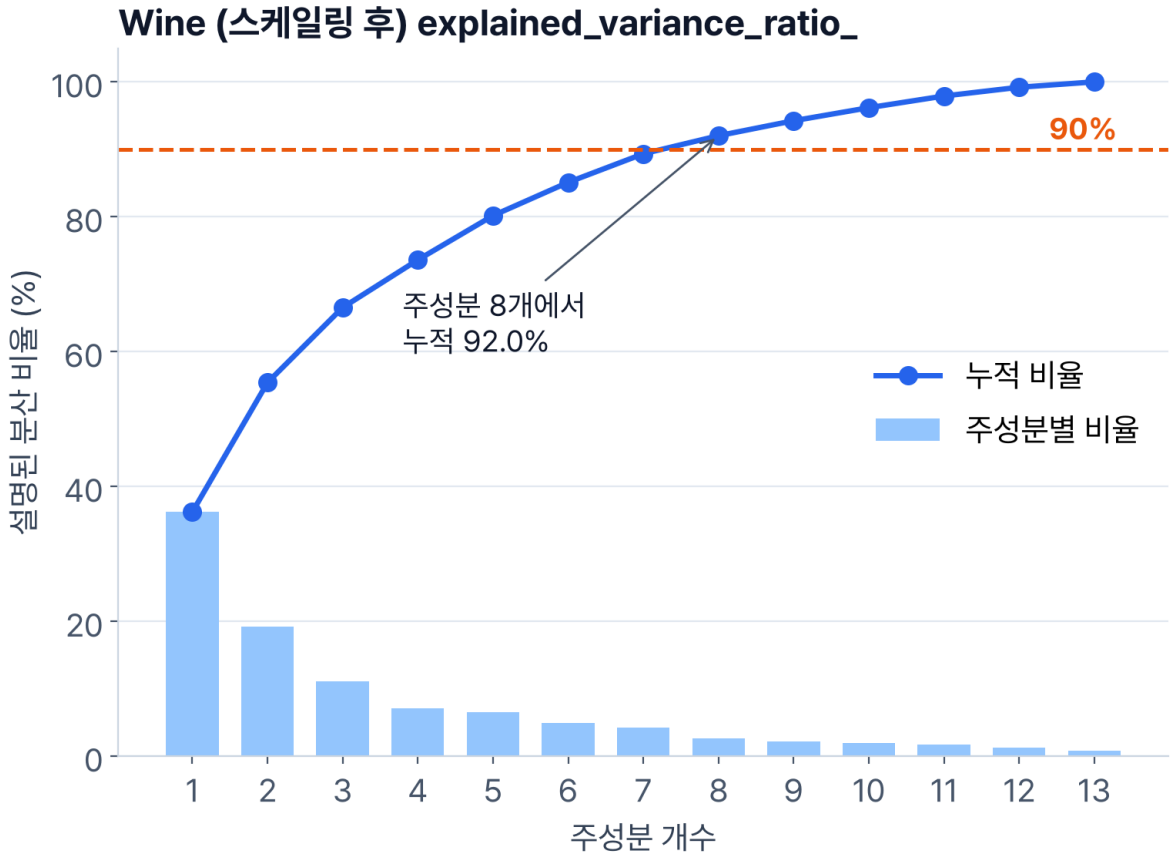
- X, y = load_wine(return_X_y=True)
- X_s = StandardScaler().fit_transform(X)
- pca = PCA(n_components=2); Z = pca.fit_transform(X_s)
- pca.explained_variance_ratio_ → [0.362, 0.192]

주성분 개수를 비율로 지정

- PCA(n_components=0.9, svd_solver="full") → 누적 90%를 넘는 최소 개수 (Wine: **8개**)
- 개수는 pca.n_components_ 로 확인

scikit-learn 1.9.1에서 실행 확인 (2026-10-13). PCA는 중심화만 하고 스케일링은 하지 않는다.

05 주성분은 몇 개 남길까



누적 설명 비율

- explained_variance_ratio_ 의 **누적합**을 본다
- 목표(예: 90%)를 처음 넘는 개수를 고른다

Wine 데이터

- 13개 피처 → 주성분 **8개**에서 누적 92.0%
- 2개만 남기면 55.4% (시각화용)

데이터: load_wine + StandardScaler, PCA(), scikit-learn 1.9.1 실측 2026-10-13

05 다른 차원 축소: t-SNE · UMAP

PCA는 선형, t-SNE · UMAP은 비선형

	PCA	t-SNE · UMAP
방식	선형: 분산이 큰 직교 축	비선형: 가까운 이웃 관계를 보존
주 용도	전처리 · 압축 · 시각화	주로 2~3차원 **시각화**
새 데이터	transform으로 같은 변환 적용	t-SNE는 transform 없음 (UMAP은 있음)
도구	sklearn.decomposition.PCA	sklearn.manifold.TSNE / **umap-learn** 패키지

정리 PCA 한 장 요약

무엇

분산이 큰 방향부터 **직교하는 새 축**(주성분)을 찾는다

각 점을 (PC1, PC2, ...) 새 좌표로 나타낸다

전처리

단위가 다르면 **스케일링 먼저**

Wine: 99.8% → 36.2%

몇 개

explained_variance_ratio_ 누적합으로 정한다

n_components=0.9 처럼 비율 지정

주의

주성분은 원래 피처를 섞은 것이라 해석이 어렵다

비선형 구조 시각화는 t-SNE · UMAP